DevOps Lab Assessment - Experiment 6

Docker Storage: Ephemeral Container Layers, Named Volumes & Bind Mounts

STUDENT NAME:	Harsh	**ROLL NUMBER:**	2301730194
EMAIL ADDRESS:	iharshsen08@gmail.com	**ASSESSMENT:**	Question 6 (5 Marks)
COURSE / LAB:	DevOps Lab 2 (Containerization & K8s)	**ENVIRONMENT:**	Docker 29 / Minikube v1.36 / kubectl

1. Experiment Objective & Problem Statement

Submitted by Harsh (Roll No: 2301730194, Email: iharshsen08@gmail.com). By default, all files created inside a container are stored on a thin, ephemeral writable layer. The objective of this experiment is to empirically demonstrate: (1) data loss occurring upon container deletion in the default storage driver, (2) permanent persistence across container lifecycles using Docker Named Volumes, and (3) bidirectional host-container synchronization using Docker Bind Mounts.

2. Storage Drivers & The Copy-on-Write (CoW) Paradigm

Docker utilizes union filesystems (OverlayFS). An image is comprised of immutable, read-only layers. When a container is instantiated, Docker adds a thin read-write Container Layer on top. Any modified or newly created file resides exclusively in this container layer. When the container is destroyed (docker rm), this layer is immediately deleted from the host disk. To persist data, Docker provides Named Volumes (managed in /var/lib/docker/volumes) and Bind Mounts (direct mapping to arbitrary host directories).

3. Experiment 6.1: Demonstration of Data Loss in Ephemeral Container Layer

```
$ # 1. Launch ephemeral container without external volume mounts docker run -d --name temp-worker
alpine:latest sleep 3600
# 2. Write critical state data to container layer (Candidate: Harsh) docker exec temp-worker sh -c "echo
'CANDIDATE_2301730194_STATE_DATA' > /tmp/state.txt" docker exec temp-worker cat /tmp/state.txt # 3.
Delete the container
docker rm -f temp-worker # 4. Launch new container from identical image docker run -d --name temp-worker-2
alpine:latest
sleep 3600 docker exec temp-worker-2 cat /tmp/state.txt

temp-worker (running) CANDIDATE_2301730194_STATE_DATA temp-worker (deleted) temp-worker-2 (running) cat:
can't open
'/tmp/state.txt': No such file or directory OBSERVATION: File /tmp/state.txt was completely destroyed
with the container
layer.
```

4. Experiment 6.2: Permanent Storage Using Docker Named Volume

```
$ # 1. Create Docker named volume docker volume create harsh-app-vol docker volume inspect harsh-app-vol
--format
"Mountpoint: {{.Mountpoint}}" # 2. Run Producer container and write persistent database file docker run -d
--name
vol-producer -v harsh-app-vol:/storage alpine sh -c "echo 'PERSISTED_DATA_FOR_2301730194' >
/storage/db.dat" # 3.
Forcefully delete Producer container docker rm -f vol-producer # 4. Run Consumer container mounting the
same volume and
verify data docker run --rm -v harsh-app-vol:/storage alpine cat /storage/db.dat

harsh-app-vol Mountpoint: /var/lib/docker/volumes/harsh-app-vol/_data vol-producer (deleted)
PERSISTED_DATA_FOR_2301730194 OBSERVATION: Data survived container deletion and was successfully read by
a new container!
```

5. Engineering Observations & Conclusion

Key Technical Takeaways

- Author: Experiment executed by Harsh (Roll No: 2301730194).
- Data Fragility: The ephemeral container layer must never be trusted for stateful workloads.
- Volume Portability: Docker Named Volumes isolate host filesystem details, making them the standard choice for containerized databases.